



EOSDIS

NASA'S EARTH OBSERVING SYSTEM
DATA AND INFORMATION SYSTEM

Cloud-optimized access to HDF5/NetCDF4 files using the Zarr API and the OPeNDAP DMR++ Earthdata Cloud Services

Patrick Quinn

EED-3 / Element 84

patrick@element84.com

James Gallagher

EED-3 / OPeNDAP

jgallagher@opendap.org

This work was supported by NASA/GSFC under Raytheon Technologies contract number 80GSFC21CA001.
This document does not contain technology or Technical Data controlled under either the U.S. International Traffic
in Arms Regulations or the U.S. Export Administration Regulations.

2021 Fall AGU

Let's step waaaaaaay back

I have a file location. How do I get the data I care about from it?

1. **Locate** parts of the file I need to read to get the data I care about
2. **Read** the parts of the file that have my data
3. **Translate** bytes I got into the data I wanted, usually decompressing and throwing away things I had to read but didn't want

Files on a hard drive

- Latency is very low, under 0.1ms for SSD
- Throughput is very high, > 200 MB/s.
- Parallelism is low, usually one CPU

Consequences for file formats:

- Optimal formats favor many small reads getting only the necessary data.

Files in the cloud

- Latency is very high, 70ms or more for S3
- Throughput is very high, up to 25G/s
- Parallelism is very high

Consequences for file formats:

- Optimal formats emphasize using fewer reads, **even if it means fetching extra data only to discard it.**
Too many reads makes it faster to download the whole file and process offline.

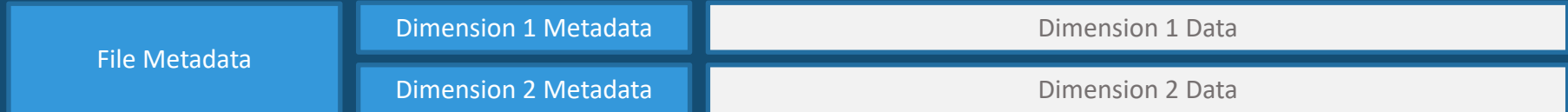
Cloud-Optimized =

Able to fetch lots of data in a small number of reads

But operating at a scale where you can't just fetch everything

Easy Win: Metadata in one read

Typical NetCDF4 (Network Common Data Form) Data File



Variable A

Variable A Metadata



Variable B

Variable B Metadata



OPeNDAP DMR++

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Variable A Metadata

Variable B Metadata

Variable C Metadata

Variable D Metadata

Byte Offsets of Chunks of Data

Typical NetCDF4 (Network Common Data Form) Data File

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Dimension 1 Data

Dimension 2 Data

Variable A

Variable A Metadata



Variable B

Variable B Metadata



The Hard Work: Data Locality

Easy Case: latitude, longitude, time. 3 Dimensions.

On disk: Always 1 dimension.

You must collapse your data dimensions to 1D. There are fundamental trade-offs that no one can solve for you.

Optimizing for analysis on one dimension hurts analysis on the others.

The Hard Work: Data Locality

“Chunking”, some extreme examples:

- Pangeo recommends 100-megabyte chunks, roughly equal dimension sizes for in-cloud processing, mostly for huge model datasets. Huge minimum data transfer but low request count.
- GEDI (Global Ecosystem Dynamics Investigation) and ICESat-2 (Ice, Cloud and land Elevation Satellite) often have ~10-kilobyte chunks for granules approximately 1 gigabyte in size. Huge request count but low minimum data transfer.

Even if they were in the same format, these would perform very differently on different systems for different use cases. Your mileage may vary!

We provide this

Typical NetCDF4 (Network Common Data Form) Data File

File Metadata

Dimension 1 Metadata

Dimension 1 Data

Dimension 2 Metadata

Dimension 2 Data

Variable A

Variable A Metadata

Variable A Data

Variable B

Variable B Metadata

Variable B Data

We provide this

I want this

Typical NetCDF4 (Network Common Data Form) Data File

File Metadata

Dimension 1 Metadata

Dimension 1 Data

Dimension 2 Metadata

Dimension 2 Data

Variable A

Variable A Metadata



Variable B

Variable B Metadata

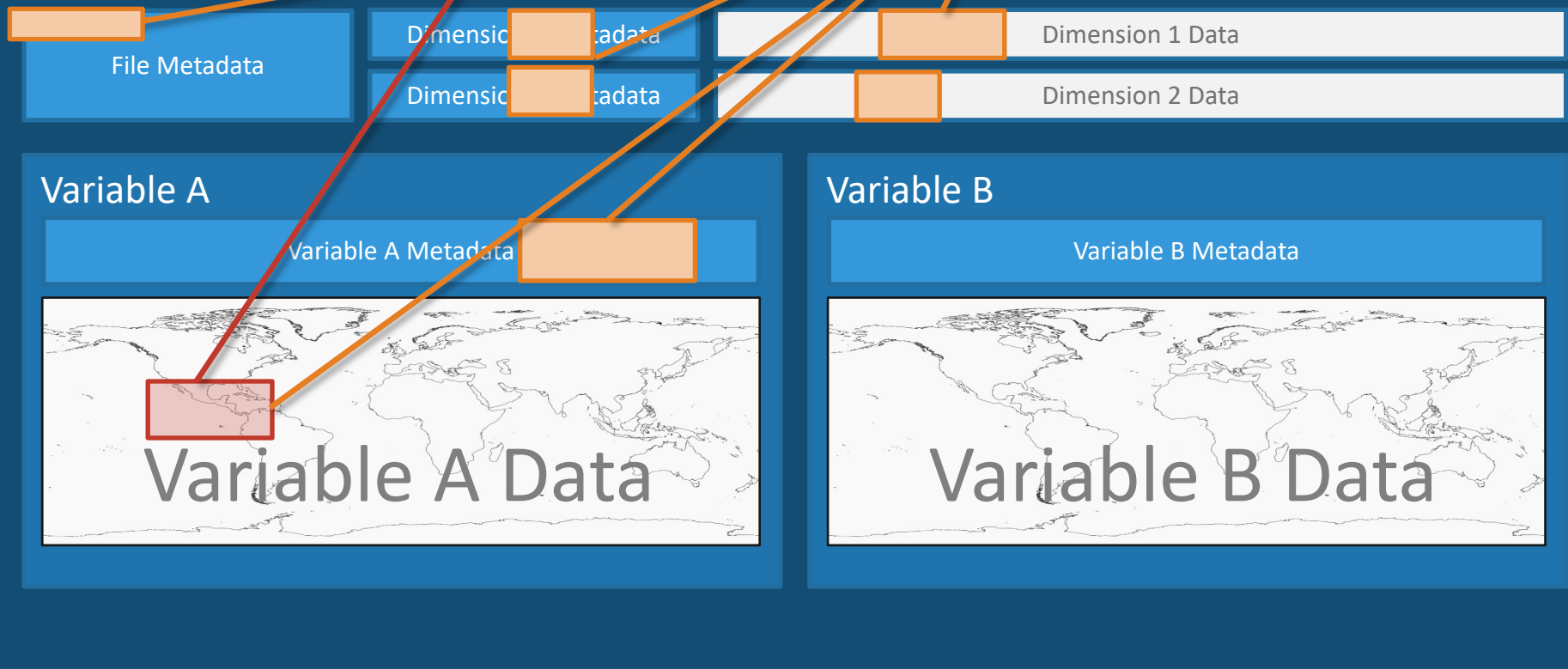


We provide this

I want this

To get it, libraries need this

Typical NetCDF4 (Network Common Data Form) Data File

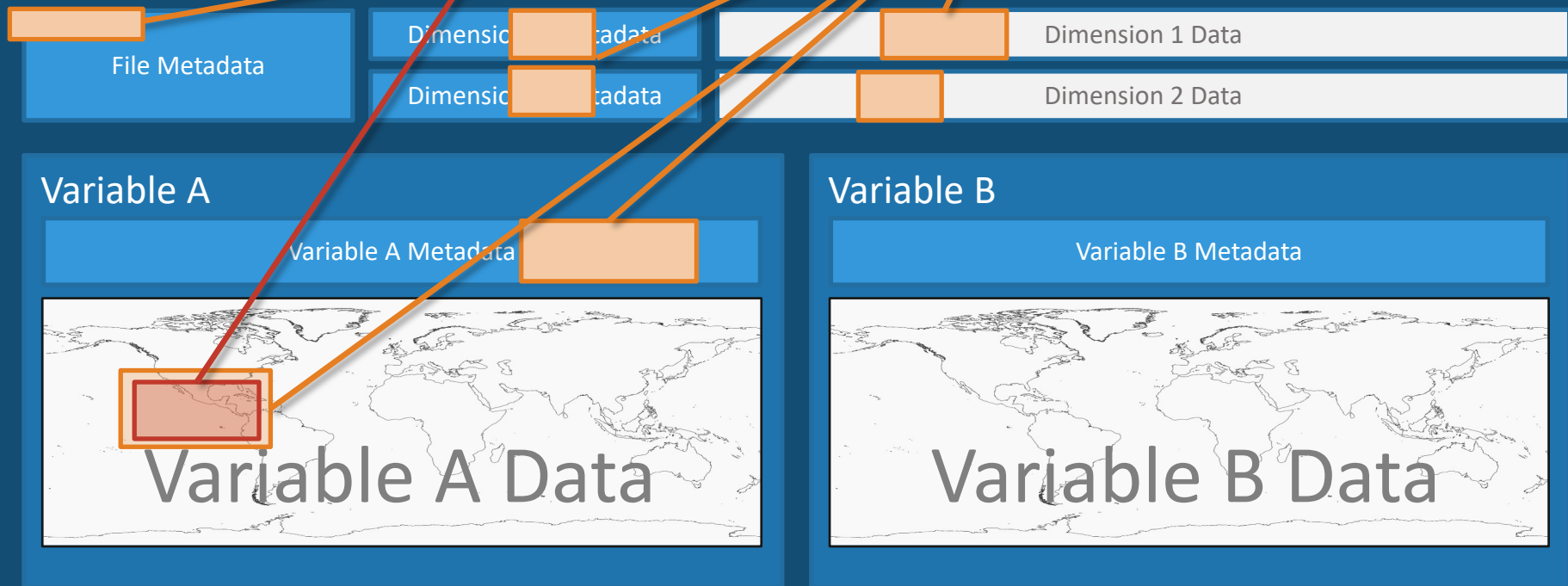


We provide this

I want this

To get it, libraries need this

Typical NetCDF4 (Network Common Data Form) Data File



The Challenge:

We need a small percentage of the file, but we don't know where to find the bytes we need without reading a significant percentage of the file.

Zarr Metadata

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Variable A Metadata

Variable B Metadata

Variable C Metadata

Variable D Metadata

Byte Offsets of Chunks of Data
(USGS + The HDF Group Prototype)

Typical NetCDF4 (Network Common Data Form) Data File

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Dimension 1 Data

Dimension 2 Data

Variable A

Variable A Metadata



Variable B

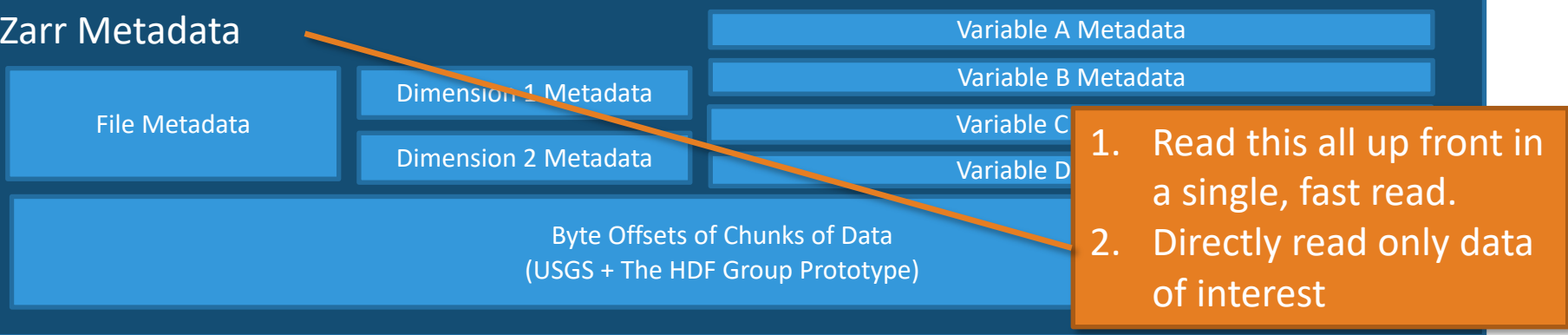
Variable B Metadata



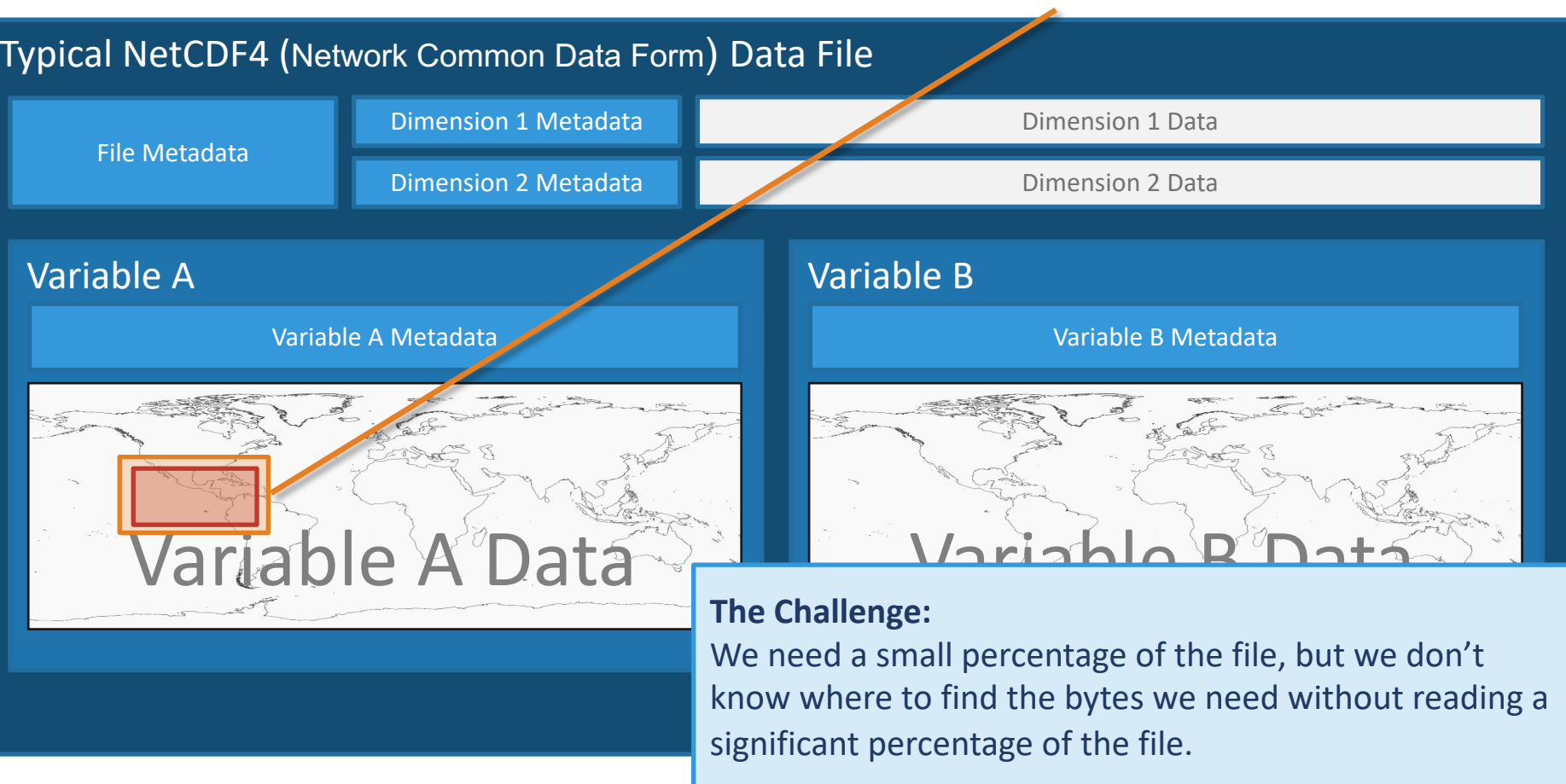
The Challenge:

We need a small percentage of the file, but we don't know where to find the bytes we need without reading a significant percentage of the file.

Zarr Metadata



Typical NetCDF4 (Network Common Data Form) Data File



How the server reads/subsets data from S3

OPeNDAP in the Cloud and the DMR++

DMR++: Dataset Metadata Response with extensions
S3: Simple Storage Service
OPeNDAP: Open-source Project for a Network Data Access Protocol

OPeNDAP DMR++

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Byte Offset Links of Data

Variable A Metadata

Variable B Metadata

Variable C Metadata

Variable D Metadata

Zarr Metadata

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Byte Offsets of Chunks of Data
(USGS + The HDF Group Prototype)

Variable A Metadata

Variable B Metadata

Variable C Metadata

Variable D Metadata

Typical NetCDF4 Data File

File Metadata

Dimension 1 Metadata

Dimension 2 Metadata

Dimension 1 Data

Dimension 2 Data

Variable A

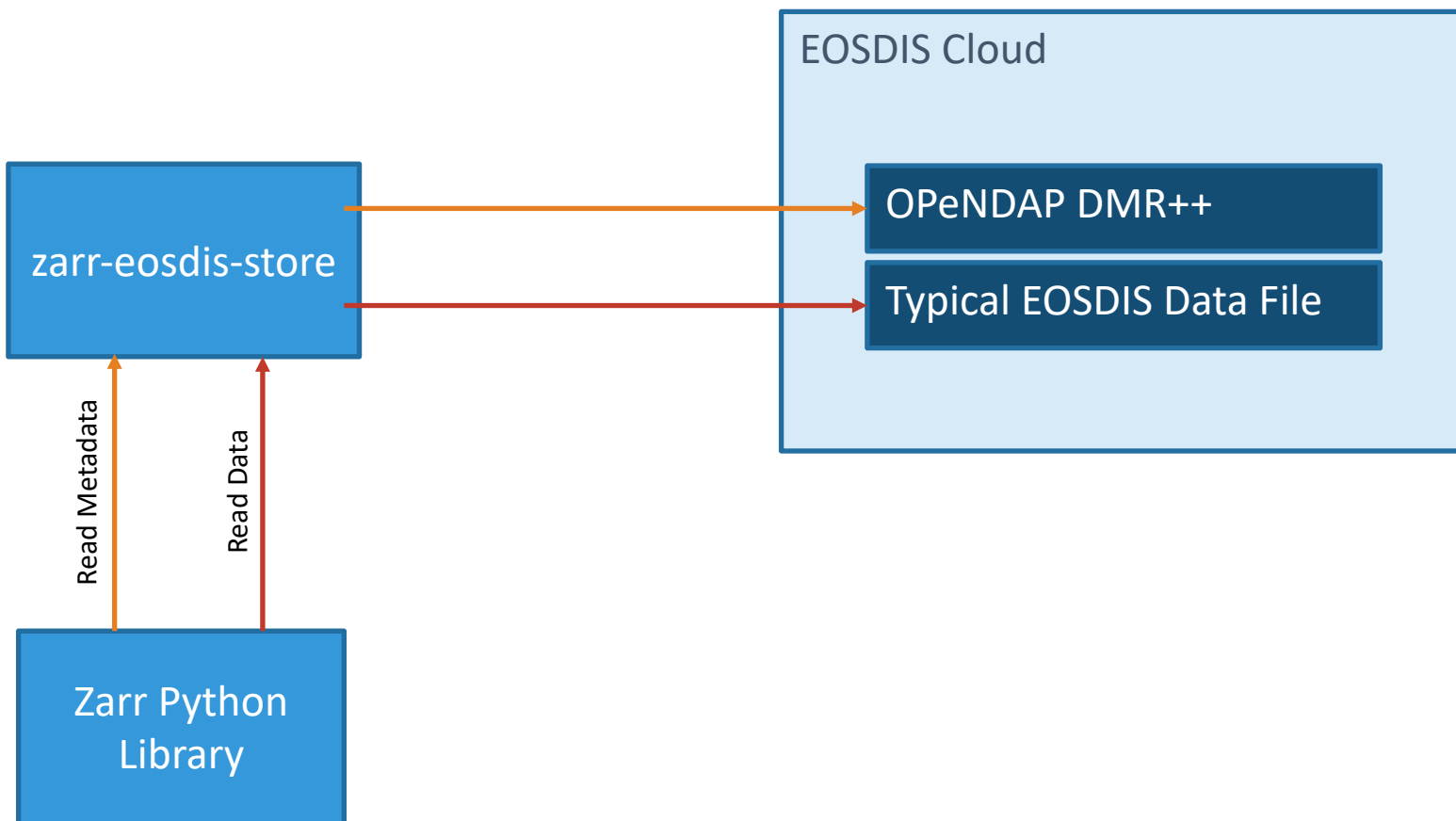
Variable A Metadata

The Challenge:

We need a small percentage of the file, but we don't know where to find the bytes we need without reading a significant percentage of the file.

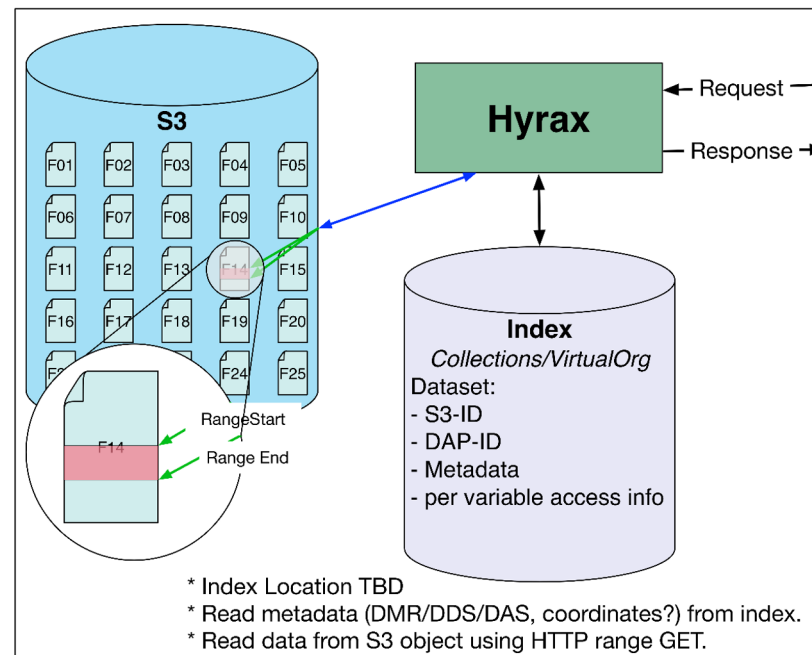


Optimization 1: Allow Reading EOSDIS NetCDF from Zarr Python



Arch 2: Files with HTTP Range GETs

HTTP Range-Get
access to whole files
in S3



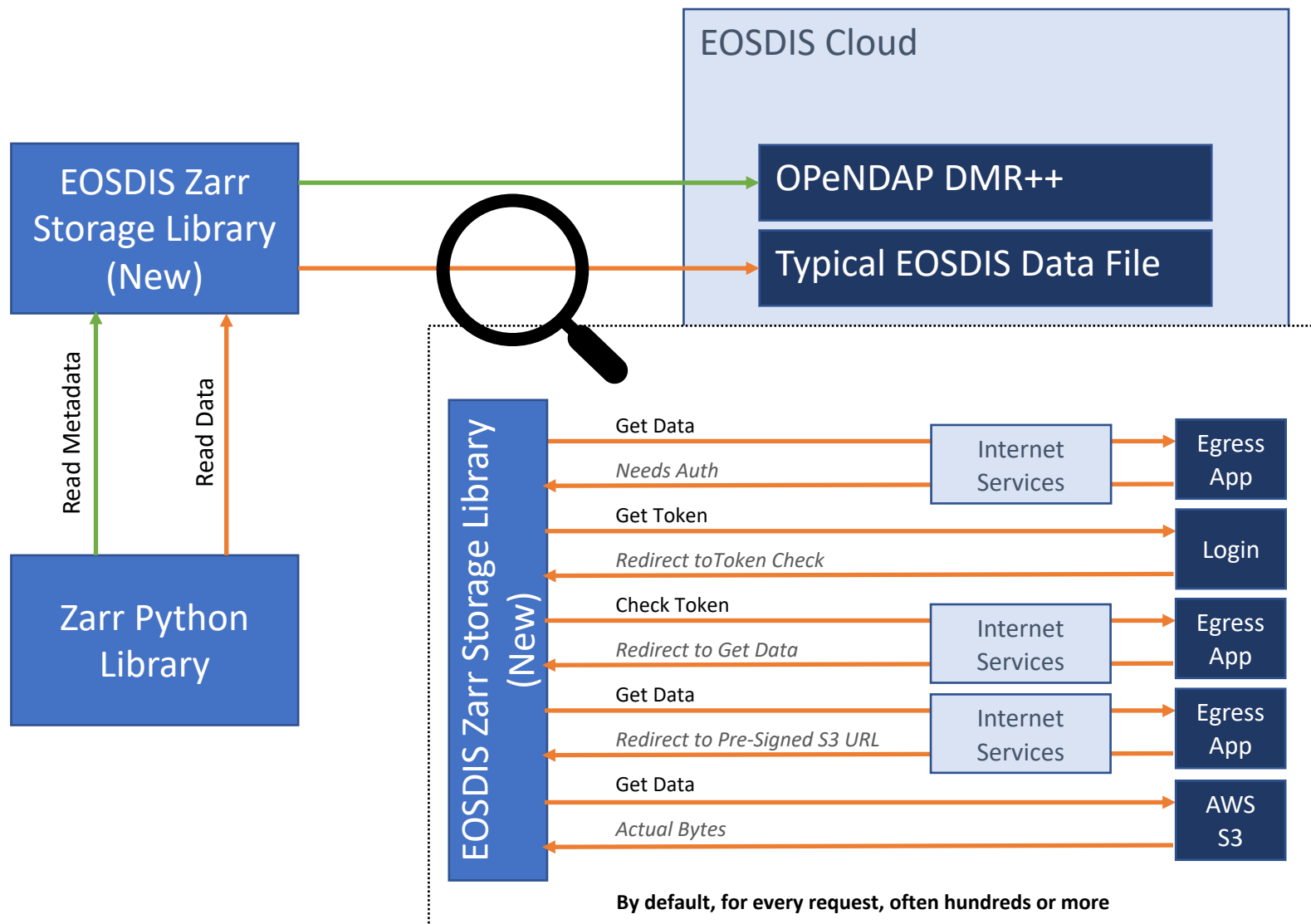
Hyrax/DMR++ Use: Set up

- First, build the DMR++ metadata file
 - Choose options that alter the appearance of the file
 - DMR++ documents are (now) XML
- Store the Data and DMR++ metadata in S3, or another web object store (WOS)
 - We use CMR to find these S3 objects

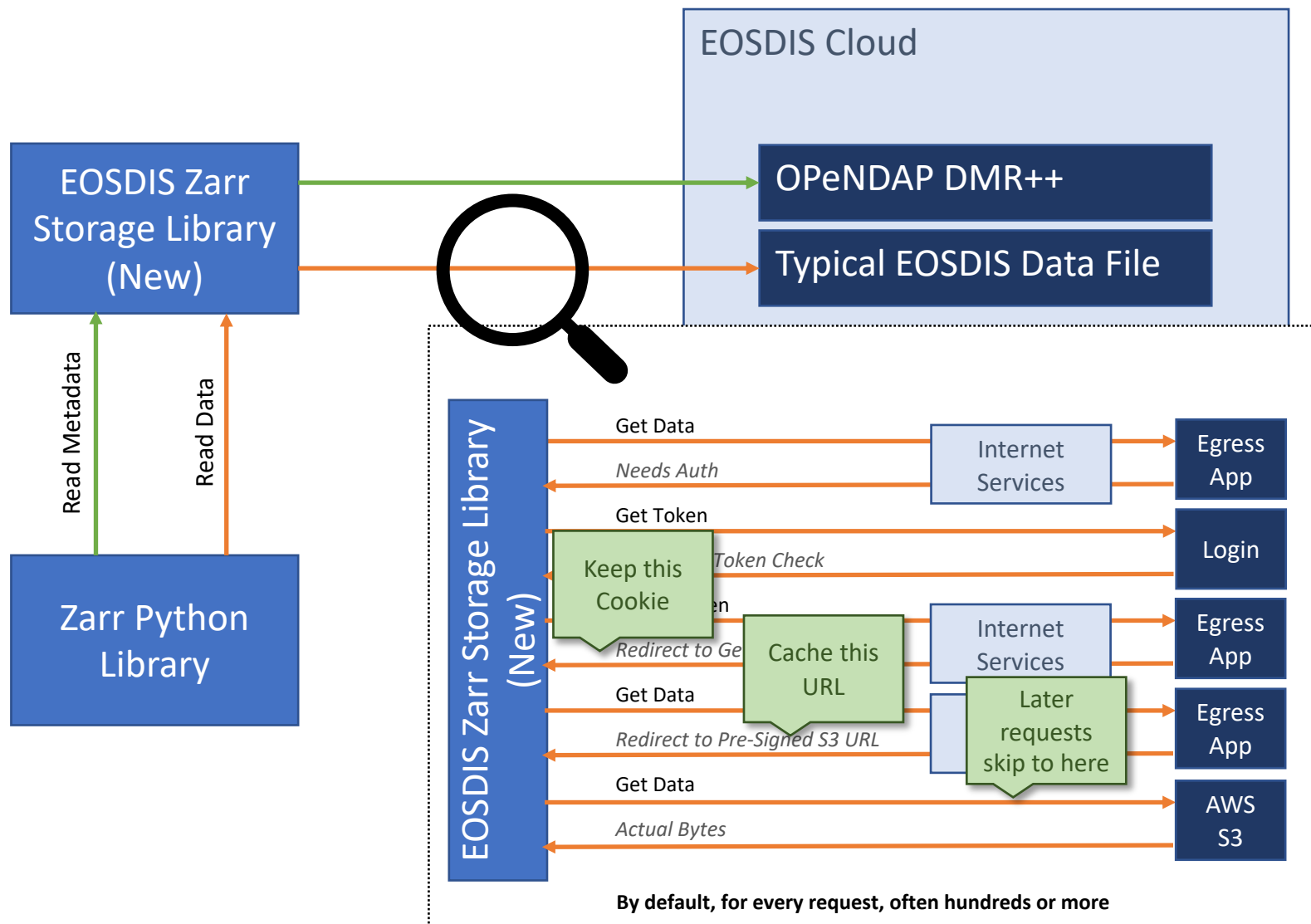
Hyrax/DMR++ Use: Optimization

- Combine adjacent 'chunks' to maximize transfer volume per request
- Issue parallel reads
- Decompress 'chunks' in parallel

Optimization 2: Cache Handshaking Information



Optimization 2: Cache Handshaking Information



It's open source!

<https://github.com/nasa/zarr-eosdis-store>

2021 Fall AGU

Conclusions

Conclusions

Cloud access optimization demands read latency optimization

1. Combine many smaller reads into fewer larger reads (metadata & data)
2. Skip expensive network actions such as auth or redirects, if possible, via cookies and caching
3. Parallelize reads

Questions?

James Gallagher

EED-3 Software Engineer

jgallagher@opendap.org

Patrick Quinn

EED-3 Software Engineer

patrick@element84.com

This work was supported by NASA/GSFC under
Raytheon Technologies contract number
80GSFC21CA001.